Tree

A few points:

So far, we have said that the tree is a non linear data structure. A tree is an acyclic, connected graph. A tree of n nodes has n -1 branches or edges. If we add an edge, a cycle is formed. If we remove an edge, the connectivity between every pair of nodes is lost.

A general tree is defined as

a) Either empty or
b) Has a root followed by number of subtrees.

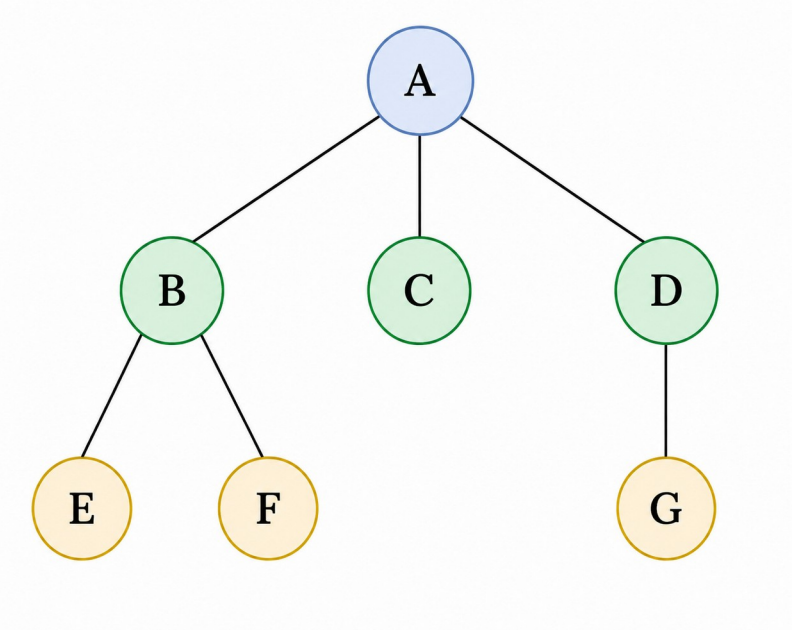
Some of the terms we use with the tree are:

Root, nodes, internal nodes, leaves, branches, path, ancestor, descendant

Parent, child, grand parent, grandchild, siblings, uncle

Height, depth, level.

Degree of a node.



A possible ADT for a general tree :

- Create a tree
- Destroy the tree
- Check empty
- Check full
- Add a node
- Remove a node
- Traverse a tree
- Find root
- Find children
- Check whether leaf
- Find height of the tree
- Find the path from the root to a given node

Binary Tree:

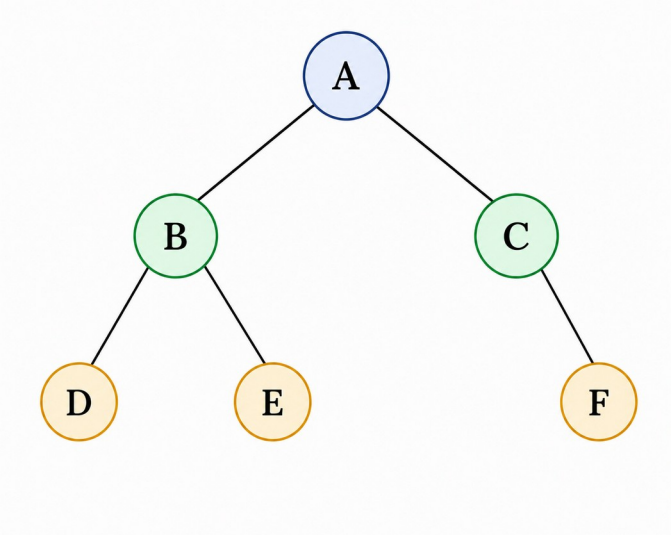
A **binary tree** is a tree in which every node has **at most two children**. The two children are distinguished as the **left child** and **right child**.

A binary tree is either:

- An **empty tree**, or
- A root node together with two disjoint binary trees called its:
    - **left subtree**
    - **right subtree**

Please observe the recursive nature of this definition. As the data structure is inherently recursive, writing recursive functions to support the interface is easy.

An example :



- A is the root.
- B is the left child of A.
- C is the right child of A.
- D and E are children of B.
- F is the right child of C.
- D, E, and F are leaves.

Binary Tree : Abstract Data Type

These are possible operations on a binary tree.

Create()
Create an empty tree
Destroy()
Remove the tree
root()
Return the root of the tree.
parent(p)
Return the parent of node p.
left(p)
Return the left child of p.
right(p)

Return the right child of p.
isLeaf(p)
Test whether p is a leaf.
component(p)
Return the component stored at p.
addRoot(x)
Create a root containing x.
addLeft(p, x)
Add x as the left child of p.
addRight(p, x)
Add x as the right child of p.
remove(p)
Remove node p and its subtree.
size()
Return the number of nodes.
Is_empty()
Test whether the tree is empty.
Is_full()
Check whether a component can be added
height()
Return the height of the tree.

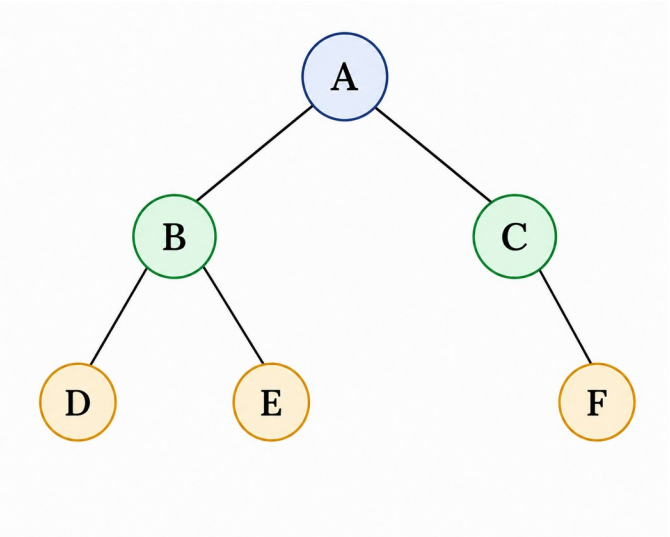
We may implement a few of these interfaces in the next sections.

Traversals:

1. Inorder traversal of a binary tree:

We can visit all the components in an array or a list from one end to another. It is not possible to do so in case of a tree. One of the ways of visiting all the components in a binary tree is to visit the left subtree somehow and then the root and then the right subtree somehow. If the tree is

empty, there is nothing to visit. As the left and the right subtrees are trees, we could apply this logic recursively.



Before visiting A, we visit left sub tree of A – that is the subtree B.

Before visiting B, we visit left sub tree of B – that is the subtree D.

Left sub tree of D is empty. So **visit D**.

Right sub tree of D is empty. So the sub tree D has been visited.

Left sub tree of B has been visited. So visit B.

Now visit the right sub tree of B.

Next we **visit E.**

Now, the left sub tree of A is visited. So **visit A** and then its right subtree.

This results in **visit C** and then **visit F.**

So the traversal of this tree will result in :

D B E A C F

So, in inorder traversal of a binary tree,

If the tree is not empty

We visit the left subtree in inorder

Then the root

Then visit the right subtree in inorder

2. **preorder traversal of a binary tree:**

   In this traversal,

   If the tree is not empty,

   We visit Then the root

   Then left subtree in preorder

   Then visit the right subtree in preorder

   For the tree above, the preorder traversal will result in the following listing.

   A B D E C F

3. **postorder traversal of a binary tree:**

   In this traversal,

   If the tree is not empty,

   We visit left subtree in postorder

   Then visit the right subtree in postorder

   Then the root

   For the tree above, the postorder traversal will result in the following listing.

   D E B F C A

Implementation of Binary Search Tree:

To start with, we will make a simple example of binary trees. We will use a special binary tree called binary search tree.

In a binary search tree, every component has a key. The components are arranged based on the key.

If the tree is empty, the component is put at the root.

Otherwise, if the key in the component at the root is higher than the key in the new component, we will insert in the left subtree.

Otherwise, we will in the right subtree.

You will observe that the insertion always happens as a leaf.

One of the implementations will be as follows.

The tree has a root. This is like a list having a head.

The root points to a node structure.

The node structure has a component – called key – in this example.

Apart from that, it has two pointers – which point to the left and the right sub trees.

```
struct tree
{
        node_t *root_;
};
typedef struct tree tree_t;

struct node
{
        int key_;
        struct node *left_;
        struct node *right_;
};
typedef struct node node_t;
```

This is the simplest interface to start experimenting.

void init(tree_t *ptr_tree);

- Ground the root.

void deinit(tree_t *ptr_tree);

- Walk through the tree
- Remove the left subtree
- Remove the right subtree
- Remove the root

void disp(tree_t *ptr_tree); // This is inorder traversal of the tree

- If the tree is not empty
    - Display the left subtree
    - Display the root
    - Display the right subtree

void insert(tree_t *ptr_tree, int key);

- If the tree is empty make this the root
- Otherwise compare the keys
- Decide to insert in the left subtree or the right subtree

We mentioned that the tree is inherently a recursive data structure.

But the tree structure by itself is not recursive – the node structure which is self-referential is recursive. So, the functions in the tree call their corresponding recursive counter part in nodes.

Let us have a look at them.

Traversal:

Observe that the tree function disp_inorder calls disp_recursive_inorder of the node.

The disp_recursive_inorder_function of the node calls itself on its left subtree, displays the root and Calls itself on its right subtree.

```
void disp_inorder(tree_t *ptr_tree)
{
	printf("Tree inorder: ");
	disp_recursive_inorder(ptr_tree->root_);
	printf("\n");
}

static void disp_recursive_inorder(node_t* temp)
{
	if(temp)
	{
		disp_recursive_inorder(temp->left_);
		printf("%d ", temp->key_);
		disp_recursive_inorder(temp->right_);
	}
}
```

The other two traversals are similar.

Check the files in the folder : inorder-preorder-postorder.

Insert:

```
void insert(tree_t *ptr_tree, int key)
{
        node_t *temp = (node_t*)malloc(sizeof(node_t));
        temp->key_ = key;
        temp->left_ = temp->right_ = NULL;
        ptr_tree->root_ = insert_recursive(ptr_tree->root_, temp);
}
```

The above tree routine creates a node. Assumes that it succeeds in memory allocation. Populates the node. Calls the insert_recursive function of the node to return the root.

This function is very interesting. If the tree is empty, then this new node becomes the root. Otherwise based on the comparison of the key, we try to insert in the left subtree or the right subtree. If the insertion is in the left subtree, then left of the root is the root for that subtree. Similarly, if the insertion is in the right subtree, the right of the root is the root for that subtree. So, the return value of the function is assigned to the appropriate root.

You may want to draw and check how this works for a tree of at least 3 nodes.

```
static node_t* insert_recursive(node_t* root, node_t* temp)
{
	if(root == NULL)
	{
		root = temp;
	}
	else if(root->key_ > temp->key_)
	{
		root->left_ = insert_recursive(root->left_, temp);
	}
	else
	{
		root->right_ = insert_recursive(root->right_, temp);
	}
	return root;
}
```

Deinitialize:

Like other functions, deinit of tree calls the deinit_recursive of node.

```
void deinit(tree_t *ptr_tree)
{
	deinit_recursive(ptr_tree->root_);
	ptr_tree->root_ = NULL;
}
```

This recursive function on the node clears the left subtree recursively, then the right subtree recursively and then the root.

Please node that the order matters.

```
static void deinit_recursive(node_t* root)
{
	if(root)
	{
		deinit_recursive(root->left_);
		deinit_recursive(root->right_);
		free(root);
	}
}
```

Adding a few more functions in the interface of the tree:

The following functions indicate how we formulate recursive routines on binary search trees.

Most of these functions use the simplest model of a binary tree :

- Empty
- Root followed by left and right subtrees

a) Count the number of nodes

If the root is NULL, return 0

Else return 1 + number of nodes in the left subtree + number of nodes in the right subtree

b) Count the number of leaves

If the root is NULL, return 0

Else if the left subtree and the right subtree of the root are empty, return 1

Else return number of leaves in the left subtree and the number of leaves in the right subtree.

c) Check whether the two trees say T1 and T2 are same

If T1 and T2 are empty, then return true

Else if T1 or T2 is empty, then return false

Else return key of root of T1 = key of root of T2 and left subtree of T1 = left subtree of T2 and right subtree of T1 = right subtree of T2

Please check the folder copy_and_compare for the code.

```
// node
int are_same_recursive(node_t* root1, node_t* root2)
{
        if(root1 == NULL && root2 == NULL)
                return 1;
        else if(root1 == NULL || root2 == NULL)
                return 0;
        else
                return root1->key_ == root2->key_ &&
                        are_same_recursive(root1->left_, root2->left_) &&
                        are_same_recursive(root1->right_, root2->right_);
}
// tree
```

```
int are_same(tree_t* ptr_tree1, tree_t* ptr_tree2)
{
        return are_same_recursive(ptr_tree1->root_, ptr_tree2->root_);
}
```

d) Copy one tree to another (say T1 to T2)

If T1 is empty, make T2 empty

Else

Make a node

Copy the key of root of T1 to the new node

Copy the left subtree of T1 to left subtree of the new node created

Copy the right subtree of T1 to right subtree of the new node created

Return the new node

Please check the folder copy_and_compare for the code.

```
// node
node_t* copy_recursive(node_t* ptr_src)
{
    node_t *temp;
    if(ptr_src != NULL)
    {
        temp = (node_t*)malloc(sizeof(node_t));
        temp->key_ = ptr_src->key_;
        temp->left_ = copy_recursive(ptr_src->left_);
        temp->right_ = copy_recursive(ptr_src->right_);
        return temp;
    }
    else
    {
        return NULL;
    }
}
```

```
// tree
void copy(tree_t* ptr_src, tree_t* ptr_dst)
{
    ptr_dst->root_ = copy_recursive(ptr_src->root_);
}
```

Observe the code is somewhat similar to that of insert.

You may want to convince yourself why the copy function is void, but copy_recursive returns a pointer to a node.

Search for a key in a binary search tree:

A binary search tree is organized based on a predicate applied at the root. We use this predicate while inserting as well as searching.

While inserting if the root is empty, the node with the key is made the root.

Otherwise, we compare the key with the key at the root. If the key is less than the key at the root, we insert in the left subtree otherwise in the right subtree.

There are two aspects to searching in a tree.

- How to search?
- What to return?

If the tree is empty, the key is not found.

If the key matches the key at the root, the key is found at the root.

Otherwise if key is less than the key at the root, we search the left subtree recursively

Else search the right subtree recursively.

What should the search function return?

Is it meaningful if the search function displays a message? Absolutely NO. The client cannot access the information found. A function should output something on the screen only if it is for displaying.

Should be fine if the function returns true if the key is found otherwise false? Not really. The client may want to get the component found with respect to the key.

The right way is to have something called an iterator which represents the concept of position. But we are not discussing this in this course. So, we will return a pointer to a node which is not the best.

```
node_t* search_recursive(node_t* root, int key)

{

    if(root == NULL)

            return NULL;

    else if(key == root->key_)

            return root;

    else if(key < root->key_)

            return search_recursive(root->left_, key);

    else

            return search_recursive(root->right_, key);

    }
```

```
node_t* search(tree_t *ptr_tree, int key)
{
    return search_recursive(ptr_tree->root_, key);
}
```

The search function on the tree is non-recursive. It in turn calls the recursive function on the node.

Check the code in the folder search_delete_key

Delete a key :

This is the most complicated function on trees we are encountering so far.

Delete key on the tree is delegated to delete key recursive function on node.

```
void delete_key(tree_t *ptr_tree, int key)
{
    ptr_tree->root_ = delete_key_recursive(ptr_tree->root_, key);
}
```

There are many cases to consider.

```
node_t* delete_key_recursive(node_t* root, int key)

{

// code is here.

}
```

- If the tree is empty, then nothing to do. We cannot delete a node with a given key.

```
if(root == NULL)

{

        return NULL;

}
```

- If the key is less than the one at the root, call delete key on its left subtree

```
else if(key < root->key_)
  {
    root->left_ = delete_key_recursive(root->left_, key);
  }
```

- If the key is greater than the one at the root, call delete key on its right subtree

```
else if (key > root->key_)
{
    root->right_ = delete_key_recursive(root->right_, key);
}
```

- Now we have found the match for the key. We have consider 4 cases
    - Key at the leaf
    - Key at a node with no left subtree
    - Key at a node with no right subtree
    - Key at a node with both left and right subtree.

- Let us consider each case.
- **Key at the leaf.**

  This is the simplest case.

  ```
  if(root->left_ == NULL && root->right_ == NULL)
          {
                  free(root);
                  return NULL;
          }
  ```

  Remove the node and ground the pointer in its parent.
- **Key at a node with no left subtree**

  Attach its right subtree to its parent.

  ```
  else if(root->left_ == NULL)
          {
                  node_t* temp = root->right_;
                  free(root);
                  return temp;
          }
  ```
- **Key at a node with no right subtree**

  Attach its left subtree to its parent.

  ```
  else if(root->right_ == NULL)
          {
                  node_t* temp = root->left_;
                  free(root);
                  return temp;
          }
  ```

- **Key at a node with both left and right subtrees.**

  Find the inorder successor of what comes to the right.

```
node_t* inorder_successor(node_t* root)
{
    node_t* temp = root;
    while(temp->left_ != NULL)
        temp = temp->left_;
    return temp;
}
```

  Replace the key at the root with the successor.

  Modify the problem to remove this new key in the right subtree.

```
else
        {
            node_t* temp = inorder_successor(root->right_);
            root->key_ = temp->key_;
            root->right_ = delete_key_recursive(root->right_, temp->key_);
        }
```

  Check the code in the folder search_delete_key

How to store a tree and how to restore a tree?

If a list has to be stored, we could traverse and right the components to a file. We can restore the list by reading the components from the file and attaching them at the end of the list.

A list is a linear data structure. So we cannot do the way we have stated above.

A tree is non linear. A single traversal will not define the tree completely.

For example, if a tree of 3 nodes has an inorder traversal A B C, there are 5 possible trees which can produce the same inorder traversal.

An inorder and a postorder traversals of a tree uniquely defines a tree. Given the two traversals, we can reconstruct the tree.

Let us take an example to illustrate.



There are exactly 10 nodes:

A, B, C, D, E, F, G, H, I, J

The inorder traversal of the tree : D B H E A F C G J I

The post order traversal of the tree : D H E B F J I G C A

To reconstruct the tree, traverse the post order traversal from the right. The last node A has to be the root of the whole tree. Therefore, A is the root and A divides the inorder traversal into two parts. All nodes to the left of A are in left subtree of A and all the nodes to the right of A in the right subtree.

So, we know the root of the tree.

Next to the right of A in postorder traversal is C. C is to the right of A in inorder traversal. C is the root of the right subtree of A.

This way we may continue to build the whole tree.

The rest of it is left as an exercise for the readers.